PointPillars + nuScenes mini

LiDAR BEV 三维目标检测 · 本科实践指导书

课程版本：Windows / Anaconda / 单关键帧 / 公开权重微调检测头

1. 实验内容与目标

本实验以 nuScenes mini 的 LIDAR_TOP 点云为输入，使用 PointPillars 模型完成三维目标检测。学生先测试公开预训练权重，再用训练集真实微调检测头 5 个 epoch，最后对同一验证集比较模型结果。

```
LiDAR 单帧点云 → Pillar 编码 → BEV 伪图像 → 2D 骨干 → 3D 检测框
                        ↓ 公开权重初始化
验证基线 → 训练 1 epoch → 恢复至累计 5 epoch → 验证与 BEV 图
```

2. 本课程实际配置

项目	本实验设置
数据	nuScenes v1.0-mini；训练 323 帧、验证 81 帧（沿用 Terra 已测索引）
输入	仅当前 LiDAR 关键帧 x / y / z / intensity；不读取 sweeps
模型	MMDetection3D 1.4.0 的 PointPillars；检测头微调
感知区域	x、y 均为 [-25,25] m；BEV 网格 200 × 200
训练	单卡、batch=1、AMP；先 1 epoch，再恢复到累计 5 epoch
学习重点	环境安装、真实训练、权重保存、固定验证、BEV 展示

3. 实验要求与适用范围

推荐有 NVIDIA GPU 的 Windows 10/11 电脑，优先使用 8 GB 显存、16 GB 内存及充足磁盘空间。Terra 仅报告在 RTX 3070 Laptop 8 GB 上完成训练与二次复现，4/6 GB 显存、RTX 50 系显卡等配置没有由这份代码提供等效验证。

> **先确认数据版本** 本 RAR 当前只支持 v1.0-mini。第一次课若下载的是 v1.0-trainval 精简数据，不可改名成 mini 直接运行；需由教师另行提供 mini 或适配后的课程代码。

4. 准备课程代码与数据

教师需分发以下三类材料。上传的 PointPillarsCourse.rar 仅包含 10 个 .py 文件及 requirements-course.txt，不包含模型权重、独立完整配置、原始点云和 nuScenes 预处理信息文件。

4.1 学生创建的目录

```
D:\PointPillarsCourse\
  check_environment.py   preprocess.py   download_weights.py
  train.py               test.py         compare_results.py
  plot_training.py       course_common.py
  course_modules.py      course_visualization.py
  requirements-course.txt
  checkpoints\           (运行权重校验后生成)
  data_infos\            (运行预处理后生成)
  outputs\               (训练/测试结果)
D:\Datasets\nuscenes\
  v1.0-mini\             (scene.json 等官方元数据)
  samples\LIDAR_TOP\    (404 个关键帧点云)
  nuscenes_infos_train.pkl
  nuscenes_infos_val.pkl
D:\PointPillarsAssets\  (教师提供的权重和独立配置)
```

4.2 数据与模型必须准备齐全

材料	来源与检查
课程脚本	解压 RAR 中全部文件到 D:\PointPillarsCourse；不要只拷 train.py。
nuScenes mini	已有 mini 可复用；数据根目录含 v1.0-mini 和 samples/LIDAR_TOP。无需 sweeps 文件参与训练。
训练/验证源索引	数据根目录额外需要 nuscenes_infos_train.pkl、nuscenes_infos_val.pkl；它们由教师在已验证环境预生成并分发。
公开 PointPillars 权重	官方 f19d00a3.pth，由教师提供；完整文件名见下。
模型配置	教师提供展开继承后的、可独立加载的官方配置 .py；原仓库短配置含 _base_，单独复制它会找不到依赖。

```
hv_pointpillars_secfpn_sbn-all_4x8_2x_nus-3d_20210826_225857-f19d00a3.pth
pointpillars_hv_secfpn_sbn-all_8xb4-2x_nus-3d.py
```

> **教师课前检查** 确认这两份模型文件来自相互匹配的官方 nuScenes SECFPN 配置；后者必须是展开后的独立配置。原始训练/验证 PKL 要与同一 v1.0-mini 的 404 帧匹配。

5. 使用 Anaconda 配置环境

打开 Windows 开始菜单 → Anaconda Prompt。后续命令均在这个 CMD 终端执行，不使用 PowerShell 的换行符。原先第一次课的 YOLO/PETR 环境不要直接混用。

5.1 创建课程环境

```
conda create -n pointpillars python=3.9 pip -y
conda activate pointpillars
python --version
nvidia-smi
```

正常情况下，提示符前出现 (pointpillars)，Python 为 3.9.x；nvidia-smi 能显示 NVIDIA 显卡。安装驱动通常即可运行带 CUDA runtime 的 PyTorch wheel，不必仅因为驱动显示 CUDA 12.x 就安装完全相同版本的 CUDA Toolkit。

5.2 安装 PyTorch 与 OpenMMLab

```
python -m pip install torch==2.0.1 torchvision==0.15.2 ^
  --index-url https://download.pytorch.org/whl/cu118
python -m pip install numpy==1.24.4 scipy==1.13.1 ^
  matplotlib==3.5.3 opencv-python==4.8.1.78
python -m pip install mmengine==0.10.7 mmdet==3.2.0
python -m pip install mmcv==2.1.0 --only-binary=mmcv ^
  -f https://download.openmmlab.com/mmcv/dist/cu118/torch2.0/index.html
python -m pip install mmdet3d==1.4.0 --no-deps
```

5.3 安装数据集与可视化依赖

```
python -m pip install nuscenes-devkit==1.1.11 ^
  numba==0.60.0 plyfile==1.1.3 scikit-image==0.24.0 ^
  tensorboard==2.14.0 trimesh==4.4.9 yapf==0.40.1
python -m pip install open3d==0.18.0
python -m pip install lyft-dataset-sdk==0.0.8 --no-deps
```

这些版本依据 Terra 的 Windows 实机依赖清单整理。安装包容量、下载速度和 GPU 兼容性随机器而变化；最好在课前提前安装，或由教师提供相同环境的离线 wheel。

> **常见安装问题** 如果 MMCV 没有找到兼容的二进制 wheel，不要让本科生直接编译 CUDA 扩展，应向教师反馈显卡、Python 和 PyTorch 版本；RTX 50 系不应直接沿用 RTX 3070 的安装结论。

6. 每次实验前检查环境与数据

6.1 设置工作目录和数据位置

重新打开 Anaconda Prompt 后，先执行下列命令；数据不在 D 盘时，仅修改第二条路径。环境变量 NUSCENES_ROOT 是当前 Terra 代码寻找数据的优先入口。

```
conda activate pointpillars
cd /d D:\PointPillarsCourse
set NUSCENES_ROOT=D:\Datasets\nuscenes
set PYTHONUTF8=1
```

6.2 验证 Python/GPU/mini 元数据

```
python check_environment.py
```

看到 ENVIRONMENT_OK 后，还要核对输出中的 cuda_available=True、v1_mini_exists=True、keyframe_bins 数量以及 data_root 是否指向你自己的数据目录。该脚本不会验证权重或源索引是否齐全，也不能单靠这条结果证明 MMCV CUDA 算子已经可用。

6.3 把教师发放的官方权重和配置复制到课程目录

先把第 4 节列出的 .pth 和展开配置 .py 放在 D:\PointPillarsAssets，然后执行：

```
python download_weights.py --source-dir D:\PointPillarsAssets
dir checkpoints
```

成功输出 WEIGHTS_OK。它会校验模型 SHA256，并在课程目录下生成 checkpoints\pointpillars_nuscenes_official.pth 与 checkpoints\official_pointpillars_config.py。此脚本不会联网自动下载权重。

6.4 生成单帧训练/验证索引

```
python preprocess.py
findstr /C:"train_samples" /C:"val_samples" ^
  /C:"overlap_samples" data_infos\manifest.json
```

必须看到 PREPROCESS_OK、train=323、val=81、overlap=0。生成 data_infos\train.pkl、val.pkl、manifest.json。源码会去除历史 lidar_sweeps，并将每个点按 x/y/z/intensity 四维输入模型；不需要 sweeps 点云文件。

> **如果预处理直接失败** 先检查 D:\Datasets\nuscenes 下是否真的存在 nuscenes_infos_train.pkl 和 nuscenes_infos_val.pkl；当前 preprocess.py 只加工已有的官方格式源索引，不会自行创建这两份文件。

7. 微调前：先测试公开模型

基线测试的意义，是把“没有在本课程数据上再训练”的公开模型作为参考。之后的微调模型必须使用同一份 val.pkl 和相同的输入、检测范围，才能比较。

7.1 执行公开权重推理

```
python test.py ^
  --checkpoint checkpoints\pointpillars_nuscenes_official.pth ^
  --output-dir outputs\baseline_compact_bev ^
  --vis-count 3 --score-thr 0.10
```

7.2 运行成功应该看到什么

终端/文件	作用
TEST_START	开始在固定 mini 验证集上推理。
TEST_OK	模型运行、评估和绘图流程已执行完成。
outputs\baseline_compact_bev\metrics_summary.json	保存 NDS、mAP 等验证指标。
outputs\baseline_compact_bev\predictions.pkl	框架输出的原始三维检测结果。
outputs\baseline_compact_bev\visualizations\	默认 3 张 BEV PNG：LiDAR 点云＋预测框。

7.3 如何理解输出

BEV 是鸟瞰图：原始点云投影在 x–y 平面，模型预测的目标用有朝向的矩形底面框和类别/置信度表示。Terra 给出的参考基线约 NDS=0.0269、mAP=0.0001；这是单帧、50m×50m 教学范围下的验证值，不代表公开权重的完整 nuScenes 榜单精度。

学生应保存 baseline 的 metrics_summary.json 和可视化图，以便后面跟自己生成的 epoch_5.pth 比较。

> **不要误判** 本代码的 BEV 绘图只显示点云和预测框，没有叠加 ground truth 真值框。评分使用验证集标注，但图上没有标注不代表数据没有真值。

8. 训练第 1 轮并证明参数真实更新

8.1 在公开权重上微调检测头

执行下面这条训练命令。脚本会加载公开模型权重，冻结 Pillar 编码器、骨干网络和 neck，训练检测头的分类、框回归及方向相关参数；batch=1，采用混合精度（AMP）。

```
python train.py --epochs 1 ^
  --work-dir outputs\run1_compact --amp
```

训练开始后，终端出现 TRAIN_START、Epoch(train)、loss 等信息。Terra 在 RTX 3070 Laptop 8GB 上报告约 45 秒完成一轮；具体耗时请以学生自己的电脑为准。

8.2 检查训练证据

```
type outputs\run1_compact\training_audit.json
dir outputs\run1_compact\epoch_1.pth
```

字段 / 输出	判断条件
TRAIN_OK PARAMETERS_UPDATED	训练脚本成功完成审计。
validation_status	必须等于 PARAMETERS_UPDATED。
changed_parameter_tensors	应当大于 0。Terra 机器为 6/6。
gradient_parameter_tensors	应当大于 0，表明真实计算梯度。
all_trainable_parameters_finite	必须为 true，不能出现非有限参数。
epoch_1.pth	已生成自己的第 1 轮检查点。

8.3 绘制训练损失

```
python plot_training.py --work-dir outputs\run1_compact
```

成功输出 PLOT_OK，并保存 outputs\run1_compact\loss.png。损失曲线反映优化过程，不等于模型在验证集上的检测精度。

> **训练失败处理** 若出现 CUDA out of memory，先关闭占 GPU 的程序，确认已写 --amp。不要通过减少 epoch 期待降低单次反向传播的峰值显存；显卡条件不足时申请课程共享 GPU。

9. 断点恢复到累计 5 轮

第 1 轮成功后，在全新的结果目录中继续训练。这里的 --epochs 5 是最终总轮数，不是“额外再训练 5 轮”。

```
python train.py ^
  --resume-from outputs\run1_compact\epoch_1.pth ^
  --epochs 5 --work-dir outputs\run5_compact --amp
python plot_training.py --work-dir outputs\run5_compact
```

检查日志应从第 2 轮开始恢复，并生成 outputs\run5_compact\epoch_5.pth、training_audit.json 和 loss.png。Terra 在 RTX 3070 Laptop 8 GB 上报告续训约 2 分钟。

10. 加载自己的权重完成验证

```
python test.py ^
  --checkpoint outputs\run5_compact\epoch_5.pth ^
  --output-dir outputs\finetuned_compact ^
  --vis-count 3 --score-thr 0.10
```

看到 TEST_OK 后，打开 outputs\finetuned_compact\metrics_summary.json 和 visualizations 下的 BEV PNG，先检查模型确实画出了预测框，再看数值变化。

11. 自动比较微调前后的指标

```
python compare_results.py ^
  --baseline outputs\baseline_compact_bev ^
  --finetuned outputs\finetuned_compact ^
  --run1 outputs\run1_compact ^
  --run5 outputs\run5_compact ^
  --output outputs\comparison.json
```

成功输出 COMPARISON_OK。该标志仅代表比较文件成功生成，并不保证微调一定提升精度。Terra 的参考值：NDS 约 0.0269→0.1184，mAP 约 0.0001→0.0614；不同设备或运行结果可能不完全一致，必须如实记录。

> **比较规则** 两次测试必须使用同一份 mini 验证清单。不得用训练集做验证，也不能把本教学配置的 NDS/mAP 当作完整 nuScenes 官方榜单。

12. 查看、整理与提交实验结果

12.1 再次绘制不同置信度的检测图

只有模型完整测试后才可用 --vis-only，它会复用既有 predictions.pkl 生成图像，不会重新执行神经网络。由于输出目录相同，新一次重绘会覆盖同名 BEV 图片；如需比较，先把原图另存一份。

```
python test.py --vis-only ^
  --checkpoint outputs\run5_compact\epoch_5.pth ^
  --output-dir outputs\finetuned_compact ^
  --vis-count 3 --score-thr 0.30
```

0.10 与 0.30 只是显示分数阈值不同。阈值提高，通常画出的框数量减少，但模型权重和已经计算的 mAP、NDS 均不会因此改变。

12.2 最终提交清单

需要提交的文件	说明
data_infos\manifest.json	训练/验证帧数、token 无重叠、单关键帧输入依据。
outputs\run1_compact\training_audit.json	证明一轮训练有真实梯度与参数更新。
outputs\run1_compact\loss.png	第 1 轮损失图。
outputs\run5_compact\epoch_5.pth	学生自己训练的最终模型权重。
outputs\run5_compact\training_audit.json、loss.png	续训审计与损失曲线。
outputs\baseline_compact_bev\metrics_summary.json	公开模型的验证指标。
outputs\finetuned_compact\metrics_summary.json	微调模型的验证指标。
outputs\finetuned_compact\visualizations*.png	微调后的 LiDAR BEV 检测图。
outputs\comparison.json	前后验证指标变化与审计汇总。

12.3 报告只需记录的内容

填写 GPU 型号、数据划分、训练轮数和耗时；列出微调前后 NDS/mAP；附 1 张损失曲线和 2～3 张 BEV 结果图，并用几句话解释检测框与阈值变化即可，无需长篇抄写源码。

13. 常见问题与排查

出现的问题	优先检查
ModuleNotFoundError: mmdet3d / mmcv._ext	确认激活 pointpillars 环境；检查 Torch 2.0.1+cu118 与 MMCV 2.1.0 版本及匹配 wheel。
ENVIRONMENT_OK 但 preprocess.py 失败	该环境脚本不核查 infos；检查 nuScenes 数据根目录下两份 nuscenes_infos_*.pkl。
v1.0-mini 不存在	数据根目录应指向同时包含 samples 与 v1.0-mini 的外层目录；不能把 trainval 简单改名。
download_weights.py 找不到文件	核对 --source-dir；里面须有完整官方 .pth 和指定文件名的独立配置 .py。
Config 找不到 ../_base_	源配置是官方未展开的短文件；让教师提供展开继承后的完整配置。
CUDA out of memory	关闭 GPU 进程，保留 batch=1、--amp；低显存机器改用学校共享 GPU。
审计不是 PARAMETERS_UPDATED	检查 1 轮训练日志与梯度；不能仅靠生成了 checkpoint 判断完成。
TEST_OK 但 BEV 没有框	用 0.10 阈值重绘，确认权重/模型一致；也可能只是这帧没有高分预测。
验证结果不及 Terra	先核对 323/81 清单、50 m 区域、AMP 与权重；不要人为挑选更好看的帧。

14. 已安装环境后的命令速查

```
conda activate pointpillars
cd /d D:\PointPillarsCourse
set NUSCENES_ROOT=D:\Datasets\nuscenes
set PYTHONUTF8=1
python check_environment.py
python download_weights.py --source-dir D:\PointPillarsAssets
python preprocess.py
python test.py --checkpoint checkpoints\pointpillars_nuscenes_official.pth ^
  --output-dir outputs\baseline_compact_bev --vis-count 3 --score-thr 0.10
python train.py --epochs 1 --work-dir outputs\run1_compact --amp
python train.py --resume-from outputs\run1_compact\epoch_1.pth ^
  --epochs 5 --work-dir outputs\run5_compact --amp
python plot_training.py --work-dir outputs\run5_compact
python test.py --checkpoint outputs\run5_compact\epoch_5.pth ^
  --output-dir outputs\finetuned_compact --vis-count 3 --score-thr 0.10
```

参考源码：github.com/open-mmlab/mmdetection3d（v1.4.0）；模型论文：PointPillars, CVPR 2019；nuScenes：nuscenes.org。